

TD 6 : Langage PL/pgSQL

Exercice 1 : Exécuter les fonctions suivantes et indiquer le résultat obtenu.

```
// getonetuple()
CREATE OR REPLACE FUNCTION getOneTuple() RETURNS record
LANGUAGE plpgsql
AS
$function$
    declare
        res record ;
    begin
        select into res * from client ;
        return res ;
    end;
$function$ ;
```

```
//getonetupletable()
CREATE OR REPLACE FUNCTION getOneTupleTable()
RETURNS SETOF client
LANGUAGE plpgsql
AS $function$
    begin
        return query select * from client ;
    end;
$function$ ;
```

```
// lesclients()
CREATE OR REPLACE FUNCTION lesClients() RETURNS SETOF client
LANGUAGE plpgsql
AS $function$
declare
    res client%ROWTYPE ;
begin
    for res in select * from client
    loop
        return next res ;
    end loop;
    return;
end ;
$function$ ;
```

Exercice 2 : Écrire une fonction qui retourne le nombre de clients habitant dans une ville. Le nom de la ville est un paramètre de la fonction.

Exercice 3 : Écrire une fonction qui retourne le nombre de clients débiteurs.

Exercice 4 : Écrire une fonction qui permet d'insérer un tuple dans la table client. Les valeurs des colonnes seront fournies en argument à la fonction sauf pour le numéro du client. Le numéro du client devra être calculé dans le fonction. La fonction récupère le dernier numéro et l'incrémente pour créer le nouveau numéro du client à enregistrer. Une information devra indiquer si le tuple a été bien enregistré ou pas.

Exercice 5 : Écrire une fonction nommée nb_operation_compte_mois() qui permet de calculer le nombre d'opérations pour un mois passé en argument pour un compte bien précis dont le numéro est passé en argument.

Nom fonction	nb_operation_compte_mois()
Description	La fonction nb_operation_compte_mois() reçoit en arguments d'entrée un mois sous forme numérique (1..12), une année sous forme numérique (ex 2015), un numéro de compte (numéro + type) et doit calculer le nombre d'opérations qui ont eu lieu pendant ce mois fourni sur le compte dont le numéro est fourni en argument. Attention : un compte peut aussi être la destination d'un versement, ce qui doit compter dans le nombre d'opérations concernant ce compte.

Arguments en entrée	Numéro du mois : entier ; numéro de l'année : entier ; numéro du compte : entier ; type de compte : entier
Résultat attendu	Nombre d'opérations sur le compte identifié par le type et le numéro du compte passés en arguments, pour le mois et l'année passée en argument.

Exercice 6 :

Écrire une fonction nommée `creer_id_internet()` qui crée l'identifiant et le mot de passe internet du client passé en argument.

Nom fonction	<code>creer_id_internet()</code>
Description	La fonction <code>creer_id_internet()</code> reçoit en argument le nom, le prénom du client. Elle va créer l'identifiant internet et le mot de passe du client. Par défaut, l'id_internet sera créé en concaténant la 1ère lettre du prénom en Majuscule avec le nom du client en minuscule. Le mot de passe sera identique à l'id_internet. En revanche, si un/des client(s) porte(nt) le même nom et prénom que ce nouveau client, l'id_internet créé sera composé de la 1ère lettre du prénom en majuscule, du nom du client en minuscule, et d'un numéro qui correspond au nombre de client(s) portant ce nom/prénom. Par exemple, si déjà 3 André Lambert existent, notre nouveau client aura pour id_internet Alambert3. Le mot de passe sera identique à l'id_internet.
Arguments en entrée	Nom : chaîne de caractères – Prénom : chaîne de caractères
Résultats attendus	Identifiant/mdp internet du client

Exercice 7:

Écrire une fonction nommée `creer_date()` qui insère dans la table date toutes les dates pour le mois et l'année passés en argument.

Nom fonction	<code>creer_date()</code>
Description	La fonction <code>creer_date()</code> prend en argument d'entrée un mois sous forme numérique et une année et va générer pour le mois défini toutes les dates possibles, en les insérant dans la table date.
Arguments en entrée	Mois : entier – année : entier
Résultats attendus	Insertion des dates du mois et de l'année passés en argument dans la table Date.

Exercice 8 :

Écrire une fonction nommée `creer_user_client()` qui crée un utilisateur PostgreSQL pour chaque client de la banque

Nom fonction	<code>creer_user_client()</code>
Description	La fonction <code>creer_user_client()</code> a pour but de créer un compte utilisateur PostgreSQL pour chaque client de la banque. Le nom de connexion à PostgreSQL du client sera son id_internet et son mot de passe. La fonction devra vérifier si le client n'existe pas déjà avant de le créer. La liste des utilisateurs de PostgreSQL se trouve dans la table pg_user. Pour créer un compte utilisateur PostgreSQL (nommé ROLE), l'instruction SQL est CREATE ROLE. La fonction <code>creer_user_client()</code> devra retourner le nombre d'utilisateurs créés
Arguments en entrée	aucun
Résultats attendus	Création d'un ROLE PostgreSQL pour chaque client de la banque